

Lab 06

CPU scheduling

Section 1: Summary – Self-study before entering the lab

1. CPU-I/O Burst Cycle is the fundamental model used to describe the execution of a process (or a thread) in an operating system.

It states that process execution consists of an alternating sequence of two main states:

- CPU Burst: A period of time when the process is using the Central Processing Unit (CPU) to perform computation (e.g., executing instructions, performing arithmetic, manipulating data).
- I/O Burst: A period of time when the process is waiting for or performing an Input/Output (I/O) operation (e.g., reading from a disk, waiting for network data, printing output, or waiting for a user input event).

A process typically starts with a CPU burst, followed by an I/O burst, then another CPU burst, and so on, until the final CPU burst ends with the process terminating.

Stage	Activity	Burst Type	Description
1. Startup	Initialize variables, set up program environment.	CPU Burst	Pure computation, executed entirely by the CPU.
2. Data Request	Send a request to the operating system to read a block of data from the hard drive.	CPU Burst	The brief time the CPU spends executing the system call to initiate the I/O operation.
3. Waiting for Disk	The program is suspended while the disk drive physically retrieves the data.	I/O Burst	The process is <i>waiting</i> for the slow I/O device. The CPU is released and can be allocated to another process.
4. Processing Data	Perform calculations, sort the data, or apply an algorithm.	CPU Burst	Intensive computation required to transform the input data.
5. Output Request	Send a request to the operating system to display the results on the monitor.	CPU Burst	Brief CPU time to execute the system call for the output operation.
6. Output Waiting	The process is suspended while the operating system writes the output to the screen or network.	I/O Burst	The process is <i>waiting</i> for the output device to complete the operation.

Stage	Activity	Burst Type	Description
7. Termination	Final cleanup and exit.	CPU Burst	A final, often very short, period of CPU execution.

▪ Significance for Scheduling

The concept of CPU and I/O bursts is critical for CPU scheduling algorithms:

- I/O-bound processes are those that have many short CPU bursts. They spend most of their time waiting for I/O.
- CPU-bound processes are those that have few very long CPU bursts. They spend most of their time using the CPU.

Schedulers must balance these two types of processes to maximize CPU utilization. For instance, a process waiting for I/O (I/O burst) must be removed from the CPU so that another process that is ready to compute (CPU burst) can take its place. This is what makes preemptive scheduling effective.

2. Preemptive and non-preemptive scheduling are two main categories of CPU scheduling that define when a running process can be forced to relinquish the CPU.

a. Non-Preemptive Scheduling (Cooperative)

In non-preemptive scheduling, once a process is given the CPU, it keeps it until it either terminates or voluntarily enters a waiting state (e.g., waiting for I/O).

- Key Characteristic: The operating system cannot forcibly take the CPU away from a running process. The process must yield control.
- Context Switch: A context switch only occurs when the running process is blocked or finishes.
- Advantages:
 - Simpler to implement.
 - Low overhead, as there are no frequent clock-interrupt context switches.
 - Disadvantages:
 - Can result in poor response time for short, high-priority processes if a single long-running process occupies the CPU for an extended period.
 - Leads to the convoy effect (where a short process gets stuck behind a very long one).
- Examples: First-Come, First-Served (FCFS) and the non-preemptive version of Shortest-Job-First (SJF).

b. Pre-emptive Scheduling (Interruptible)

In preemptive scheduling, the operating system can forcibly interrupt a running process and reallocate the CPU to another process.

- Key Characteristic: The running process can be stopped and moved from the Running state back to the Ready state by the OS scheduler. This is typically triggered by an interrupt (like a timer expiration or the arrival of a higher-priority process).
- Context Switch: Context switches are frequent, potentially occurring every time a time quantum expires or a new, higher-priority process arrives.

- Advantages:
 - Better throughput and response time, especially in interactive (time-sharing) systems.
 - Allows higher-priority or urgent processes to quickly access the CPU.
- Disadvantages:
 - Higher overhead due to frequent context switches.
 - Requires careful design to prevent data corruption when one process is interrupted mid-way through updating shared kernel data (known as race conditions).
- Examples: Round Robin (RR), Shortest-Remaining-Time-First (SRTF) (the preemptive SJF), and the standard version of Priority Scheduling.

3. Dispatcher is the module within the operating system that gives control of the CPU to the process selected by the CPU Scheduler. It is the component responsible for the actual execution of the context switch.

The dispatcher's functions are:

- Context Switching: It saves the state (context) of the process that was just running and restores the state (context) of the new process that has been selected to run.
- Switching to User Mode: It changes the CPU's operating mode from kernel mode (where the OS runs) to user mode (where the application runs).
- Jumping to the Correct Location: It restarts the program at the exact instruction where it last left off (i.e., jumps to the proper location in the user program to resume execution).

Dispatch Latency: is the time it takes for the dispatcher to stop one process and start another.

- This time interval represents the overhead involved in the scheduling process.
- Because the dispatcher is executed frequently (every time a scheduling decision is made), its speed is crucial.
- Dispatch latency must be kept as low as possible to minimize the performance hit of context switching.

4. Scheduling Criteria

CPU scheduling algorithms are evaluated and compared using several metrics:

- CPU Utilization: Keeping the CPU as busy as possible (Goal: Maximize).
- Throughput: The number of processes that complete their execution per time unit (Goal: Maximize).
- Turnaround Time: The total amount of time from the moment a process is submitted until it completes execution (Goal: Minimize).
- Waiting Time: The amount of time a process has spent waiting in the ready queue (Goal: Minimize).
- Response Time: The time it takes from when a request was submitted until the first response is produced (important for time-sharing systems) (Goal: Minimize).

5. Scheduling Algorithms

a. First-Come, First-Served (FCFS)

- i. Policy: The process that requests the CPU first is allocated the CPU first (FIFO queue).
- ii. Preemption: Non-preemptive.

- iii. Problem: It can result in the Convoy Effect, where a single CPU-bound process with a long burst time holds up many I/O-bound processes, leading to poor CPU utilization and high average waiting time.

b. Shortest-Job-First (SJF)

- i. Policy: Schedules the process with the shortest next CPU burst time.
- ii. Optimality: SJF is optimal because it gives the minimum average waiting time for a given set of processes.
- iii. Challenge: The difficulty is knowing the length of the next CPU burst in advance.
 - This is typically overcome by estimating the length, often using exponential averaging based on the length of previous CPU bursts.
- iv. Preemptive Version: The preemptive variant of SJF is called Shortest-Remaining-Time-First (SRTF).

c. Round Robin (RR)

- i. Policy: Each process gets a small unit of CPU time, called a time quantum (q) (usually 10–100 milliseconds).
- ii. Preemption: Preemptive. After the time quantum expires, the process is preempted and added to the end of the ready queue.
- iii. Performance:
 - It generally provides a better response time than SJF, but often has a higher average turnaround time.
 - If q is large, RR behaves like FCFS.
 - If q is very small, the overhead of context switching becomes too high. The time quantum must be large relative to the context switch time.

- d. Rule of Thumb: Ideally, 80% of CPU bursts should be shorter than the time quantum q.

6. Predicting the next CPU burst time

The next CPU burst time is predicted using a formula called **exponential averaging**. This prediction is necessary because the Shortest-Job-First (SJF) scheduling algorithm requires knowing the length of the next CPU burst, which cannot be known with certainty.

The formula for the exponential average is: $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$

Variables in the Formula

Variable	Description
τ_{n+1}	The next predicted CPU burst time (the new guess).
t_n	The measured length of the most recent (actual) CPU burst.
τ_n	The previous prediction of the CPU burst time (the old guess).

Variable	Description
α	A weighting factor, or constant, that determines the relative influence of the actual burst versus the historical prediction.

The value of α generally ranges from $[0..1]$. A common practice is to set $\alpha = 0.5$, which gives equal weight to recent history and past predictions.

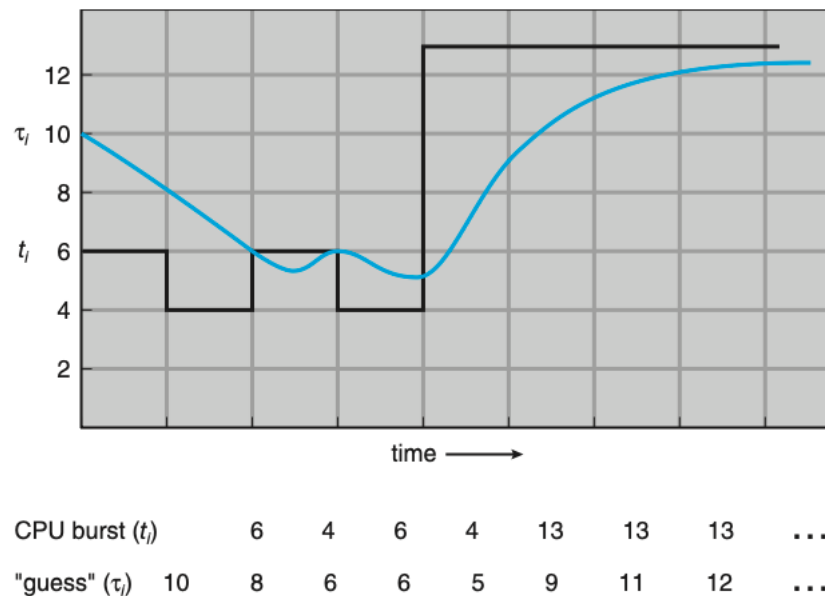


Figure 5.4 Prediction of the length of the next CPU burst.

Computation Example:

Let's use an initial prediction, τ_0 of 10 ms (milliseconds) and set the weighting factor α to 0.5.

Step 1: Calculate the second prediction τ_2

Assume the first measured actual CPU burst is $\tau_1 = 6$ ms.

$$\tau_2 = (0.5) t_1 + (1 - 0.5) \tau_1$$

$$\tau_2 = (0.5) (6 \text{ ms}) + (1 - 0.5) (10 \text{ ms})$$

$$\tau_2 = 3 \text{ ms} + 5 \text{ ms}$$

$$\tau_2 = 8 \text{ ms}$$

Step 2: Calculate the third prediction τ_3

Now, use the new prediction ($\tau_2 = 8$ ms) and assume the next actual measured burst is $\tau_2 = 4$ ms.

$$\tau_3 = (0.5) t_2 + (1 - 0.5) \tau_2$$

$$\tau_3 = (0.5) (4 \text{ ms}) + (0.5) (8 \text{ ms})$$

$$\tau_3 = 2 \text{ ms} + 4 \text{ ms}$$

$$\tau_3 = 6 \text{ ms}$$

The system would then use 6 ms as the predicted burst time for the next scheduling decision.

7. Priority Scheduling

- Policy: The CPU is allocated to the process with the highest priority. SJF is a special case of priority scheduling.
- Source of Priority: Priorities can be determined internally (by measurable quantities like time limits) or externally (by political factors like process importance).
- Problem: Starvation (indefinite blocking) of low-priority processes.
- Solution: Aging is used to gradually increase the priority of processes that wait for a long time.

8. Multilevel Queue Scheduling

- Policy: The ready queue is partitioned into several separate queues, often based on process type (e.g., real-time, interactive, batch).
- Scheduling: Each queue typically has its own scheduling algorithm (e.g., RR for foreground/interactive, FCFS for background/batch).
- Queue-to-Queue Scheduling: Scheduling *among* the queues is usually accomplished using fixed-priority pre-emptive scheduling.

9. Multilevel Feedback Queue Scheduling

- Policy: This algorithm allows a process to move between queues
- Goal: To separate processes based on the characteristics of their CPU bursts, keeping I/O-bound processes in higher-priority queues.
- Aging: A process waiting too long in a lower-priority queue may be moved to a higher-priority queue (a form of aging) to prevent starvation.
- Complexity: It is the most general and most complex scheduling algorithm, defined by parameters including the number of queues and the rules for moving processes between them.

Refer to the book for details of the multithread scheduling and Multi-processor scheduling (pages 217 to 244).

Section 2: Discussion – selected questions only (1 Hour)

1. Why is it important for the scheduler to distinguish I/O-bound programs from CPU-bound programs?
2. What is the difference between pre-emptive and non-pre-emptive scheduling?
3. Explain the difference in the degree to which the following scheduling algorithms discriminate in favour of short processes.
 - a) FCFS
 - b) RR
4. Consider the following set of processes, with the length of the CPU-burst time given in milliseconds:

Process	Burst-time	Arrival time
P1	10	0
P2	2	1

P3	3	2
P4	1	3
P5	5	4

- Draw the Gantt charts illustrating the execution of these processes using FCFS, SJN and RR (quantum=2) scheduling.
 - What is the turnaround time of each process for each of the scheduling algorithms?
 - What is the waiting time of each process for each of the scheduling algorithms?
 - Which of the schedules in Q3 results in the minimal average waiting time (over all processes)?
5. Consider the following processes with the length of a CPU burst time given in milliseconds. The processes arrived according to the arrival time. (Low numbers represent high priority)

Process	Arrival Time	Priority	Burst Time
P0	0	2	8
P5	0	1	6
P1	4	5	15
P4	9	4	13
P2	7	3	9
P3	13	1	5

- Draw 3 (three) Gantt Charts illustrating the execution of these processors using scheduling.
 - Shortest Job Next
 - Shortest Remaining Time
 - Non Pre-emptive priority scheduling
 - Pre-emptive priority scheduling
 - Round Robin (quantum=3) scheduling
- Calculate the average waiting time for Pre-emptive priority and Round Robin Scheduling

Section 3: Practical exercises

Exercise 1: [Book example] POSIX Pthreads API for setting the contention scope.

The following program listing (Figure 5.10 in the textbook) demonstrates how to configure a process's contention scope when creating threads using the POSIX Pthreads API in C.

The primary goal of this code is to show how a programmer can explicitly control the environment in which their user-level threads compete for CPU time:

- Check Current Scope:** It first uses `pthread_attr_getscope()` to determine the default scheduling scope of the system and prints whether it is `PTHREAD_SCOPE_PROCESS` or `PTHREAD_SCOPE_SYSTEM`.
- Set System Scope:** It then uses `pthread_attr_setscope()` to set the attribute to `PTHREAD_SCOPE_SYSTEM`.
- Thread Creation:** Finally, it creates a set of threads using these modified attributes.

Key Concept: Contention Scope

Setting the contention scope dictates *which* threads a newly created thread will compete against for the right to run on a physical CPU core:

- `PTHREAD_SCOPE_PROCESS` (Process-Contention Scope - PCS): In this mode, scheduling competition occurs only among threads within the same process. The operating system kernel is unaware of the individual user-level threads; it only sees the lighter-weight process entities (LWPs) provided to it by the thread library.
- `PTHREAD_SCOPE_SYSTEM` (System-Contention Scope - SCS): In this mode, scheduling competition occurs among all threads in the system. **The user-level thread is directly mapped to a kernel-level thread (LWP)**, allowing the operating system's global scheduler to manage and prioritize it against all other threads from all other processes in the entire system.

The code sets the scope to `PTHREAD_SCOPE_SYSTEM`, ensuring the threads created will be managed directly by the kernel scheduler.

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

/* Each thread will begin control in this function */
void *runner(void *param)
{
    /* do some work ... */
    pthread_exit(0);
}

int main(int argc, char *argv[])
{
    int i, scope;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    /* get the default attributes */
    pthread_attr_init(&attr);

    /* first inquire on the current scope */
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "Unable to get scheduling scope\n");
    else {
        if (scope == PTHREAD_SCOPE_PROCESS)
            printf("PTHREAD SCOPE PROCESS\n");
        else if (scope == PTHREAD_SCOPE_SYSTEM)
            printf("PTHREAD SCOPE SYSTEM\n");
        else
            fprintf(stderr, "Illegal scope value.\n");
    }
    /* set the scheduling algorithm to PCS or SCS */
    pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
```



```

/* create the threads */
for (i = 0; i < NUM_THREADS; i++)
    pthread_create(&tid[i], &attr, runner, NULL);

/* now join on each thread */
for (i = 0; i < NUM_THREADS; i++)
    pthread_join(tid[i], NULL);
}

```

1. Compile and run the program.
2. Discuss the results.

Exercise 2:[Book Example] The second program demonstrates how to configure a thread for real-time scheduling using the POSIX Pthreads API in C.

The main purpose of this code is to show how a programmer can explicitly set a thread's scheduling policy to one of the real-time standards defined by POSIX.1b. The example specifically sets the policy to `SCHED_FIFO`.

Key Functions and Concepts Demonstrated:

Function/Macro	Description
<code>pthread_attr_getschedpolicy()</code>	Used to inquire and print the current scheduling policy of the thread attributes (<code>SCHED_OTHER</code> , <code>SCHED_RR</code> , or <code>SCHED_FIFO</code>).
<code>pthread_attr_setschedpolicy()</code>	Used to explicitly set the scheduling policy for threads that will be created using these attributes.
<code>SCHED_FIFO</code> (First-In, First-Out):	A real-time policy where threads of equal priority are scheduled using FCFS. It is non-preemptive for threads of equal priority, meaning a thread will run until it yields or terminates.
<code>SCHED_RR</code> (Round Robin):	Similar to <code>SCHED_FIFO</code> , but it introduces time-slicing for threads of equal priority.
<code>SCHED_OTHER</code> :	The default, non-real-time scheduling policy.

In the provided listing, the program initializes the attributes and checks the default policy. It then attempts to set the scheduling policy for the new threads to **`SCHED_FIFO`**. By using this policy, the programmer intends for the created threads to be handled with high priority and minimal preemption relative to lower-priority threads.

```

#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

/* Each thread will begin control in this function */

```

```
void *runner(void *param)
{
    /* do some work ... */
    pthread_exit(0);
}

int main(int argc, char *argv[])
{
    int i, policy;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    /* get the default attributes */
    pthread_attr_init(&attr);

    /* get the current scheduling policy */
    if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
        fprintf(stderr, "Unable to get policy.\n");
    else {
        if (policy == SCHED_OTHER)
            printf("SCHED OTHER\n");
        else if (policy == SCHED_RR)
            printf("SCHED RR\n");
        else if (policy == SCHED_FIFO)
            printf("SCHED FIFO\n");
    }

    /* set the scheduling policy - FIFO, RR, or OTHER */
    if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0)
        fprintf(stderr, "Unable to set policy.\n");

    /* create the threads */
    for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], &attr, runner, NULL);

    /* now join on each thread */
    for (i = 0; i < NUM_THREADS; i++)
        pthread_join(tid[i], NULL);
}
```

Compile and run the code.

Exercise 3: Basic Process Scheduling Information

```
#include <stdio.h>
#include <sched.h>          // Declares sched_getscheduler()
#include <unistd.h>
#include <sys/types.h>
#include <sys/resource.h> // Required for getpriority() and PRIO_PROCESS
#include <errno.h>         // Required for error checking (errno)
#include <string.h>        // Required for strerror()

int main() {
    pid_t pid = getpid();
    int policy = sched_getscheduler(pid);
    printf("Process ID: %d\n", pid);
    printf("Current scheduling policy: ");

    switch(policy) {
        case SCHED_FIFO:
            printf("SCHED_FIFO (Real-Time)\n");
            break;
        case SCHED_RR:
            printf("SCHED_RR (Real-Time)\n");
            break;
        case SCHED_OTHER:
            printf("SCHED_OTHER (CFS)\n");

            // Correctly retrieve nice value with error check
            errno = 0;
            int nice_value = getpriority(PRIO_PROCESS, pid);

            if (errno != 0) {
                fprintf(stderr, "Error retrieving nice value: %s\n",
strerror(errno));
            } else {
                printf("Nice value: %d\n", nice_value);
            }
            break;
        case SCHED_BATCH:
            printf("SCHED_BATCH\n");
            break;
        case SCHED_IDLE:
            printf("SCHED_IDLE\n");
            break;
        default:
            printf("Unknown (%d)\n", policy);
    }

    return 0;
}
```

Compile and run the program and observe the output.

Exercise 4: CPU Affinity

CPU Affinity is a scheduling feature that allows you to bind a process (or thread) to specific CPU cores. This means the process will only run on the designated CPU(s).

Key Functions Used:

- `sched_getaffinity()` - Gets the current CPU affinity mask of a process
- `sched_setaffinity()` - Sets the CPU affinity mask for a process
- `CPU_ZERO()` - Clears all bits in the CPU set (no CPUs selected)
- `CPU_SET()` - Sets a specific CPU in the CPU set (enables it)
- `CPU_ISSET()` - Checks if a specific CPU is set in the mask

Why Use CPU Affinity?

- **Performance Optimization:** Reduces cache misses by keeping a process on the same CPU
- **Real-time Requirements:** Ensures predictable timing by avoiding CPU migration
- **Resource Isolation:** Dedicates specific CPUs to critical applications
- **NUMA Optimization:** On NUMA systems, keeps processes close to their memory

```
#include <stdio.h>
#include <sched.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    cpu_set_t mask;
    pid_t pid = getpid();

    // Get current CPU affinity
    if (sched_getaffinity(pid, sizeof(mask), &mask) == -1) {
        perror("sched_getaffinity");
        return 1;
    }

    printf("Current CPU affinity mask: ");
    for (int i = 0; i < CPU_SETSIZE; i++) {
        if (CPU_ISSET(i, &mask)) {
            printf("%d ", i);
        }
    }
    printf("\n");

    // Set affinity to CPU 0 only
    CPU_ZERO(&mask);
    CPU_SET(0, &mask);

    if (sched_setaffinity(pid, sizeof(mask), &mask) == -1) {
        perror("sched_setaffinity");
        return 1;
    }

    printf("CPU affinity set to CPU 0 only\n");

    // Do some work to see the effect
    volatile long sum = 0;
    for (long i = 0; i < 1000000000L; i++) {
```

```
        sum += i;
    }
    printf("Computation completed: %ld\n", sum);

    return 0;
}
```

Compile and run the program and observe the output.